

Tài liệu Kiến trúc và Logic: Module C++ LLM Client

Tài liệu này cung cấp cái nhìn tổng quan và phân tích chuyên sâu về cấu trúc thiết kế cũng như luồng thực thi logic của module C++ LLM Client. Module này đóng vai trò là lõi giao tiếp, hỗ trợ tương tác ổn định với máy chủ Ollama nội bộ (cụ thể là mô hình Qwen2.5) để phục vụ cho các tác vụ phân tích tài liệu và tự động sinh câu hỏi trắc nghiệm (MCQ) trong hệ thống RAG.

1. Cấu trúc Tổng quan (Architecture)

Kiến trúc hệ thống được thiết kế theo mô hình Hướng đối tượng (OOP), ưu tiên tính đa hình và khả năng mở rộng. Các thành phần được chia tách rõ ràng thành các tầng xử lý riêng biệt.

Thành phần	Vai trò & Logic Hoạt động
LLMClient (Lớp Trừu tượng)	Định nghĩa bản thiết kế chung (Interface) cho mọi Client. Chứa hàm ảo chat() và cơ chế ghi log thông qua std::function. Giúp hệ thống không bị trói buộc vào một nhà cung cấp API duy nhất.
OllamaClient (Lớp Cụ thể)	Kế thừa từ LLMClient, triển khai logic kết nối thực tế tới API của Ollama. Lưu trữ các cấu hình như Base URL, Tên Model, Temperature, Max Tokens và Timeout.
PromptType (Đầu vào)	Sử dụng std::variant để bọc hai cấu trúc dữ liệu: TextPrompt (văn bản thuần) và MultimodalPrompt (văn bản kèm hình ảnh), giúp hàm chat() chỉ cần nhận một tham số linh hoạt.

2. Phân tích Luồng Logic Cốt lõi

2.1. Logic Đóng gói Payload (Payload Builder)

Quá trình chuyển đổi từ cấu trúc C++ sang chuỗi JSON (qua thư viện `nlohmann::json`) diễn ra trong hàm `buildPayload`. Hệ thống sử dụng mẫu thiết kế Visitor pattern (thông qua `std::visit`) để phân giải loại dữ liệu đầu vào:

- **Nếu là TextPrompt:** Kiểm tra chuỗi rỗng. Nếu hợp lệ, đưa nội dung vào trường "content" của JSON.
- **Nếu là MultimodalPrompt:** Kích hoạt logic đọc file nhị phân. Dữ liệu hình ảnh được chuyển đổi thành chuỗi an toàn thông qua thuật toán **Base64 Encoding** (tự triển khai dựa trên RFC 4648 để tối ưu tốc độ), sau đó chèn vào mảng "images".

2.2. Logic Giao tiếp Mạng (Networking)

Hệ thống sử dụng thư viện `cpr` để mở một HTTP POST request tới endpoint `/api/chat`. Logic thiết lập tham số mạng rất nghiêm ngặt:

- Ép kiểu Content-Type chuẩn: `application/json`.
- Truyền Payload đã đóng gói (Serialization) trực tiếp vào Body.
- Thiết lập cờ Timeout (mặc định 60,000 ms) để tránh tình trạng hệ thống bị treo vô thời hạn khi tài nguyên suy luận của mô hình đang bận.

2.3. Logic Quản lý Lỗi Hiện đại (Modern Error Handling)

Thay vì sử dụng ngoại lệ (Exceptions) vốn tiêu tốn tài nguyên và dễ làm crash chương trình, hệ thống áp dụng chuẩn C++23 `std::expected` để quản lý trạng thái kết quả. Quy trình bắt lỗi phân cấp 3 tầng:

1. **Tầng Transport (Mạng lưới):** Bắt các mã lỗi như `OPERATION_TIMEDOUT` (Quá thời gian) hoặc `COULDNT_CONNECT` (Server chưa bật). Trả về enum `LLMErrorKind` tương ứng.
2. **Tầng HTTP (Giao thức):** Nếu server phản hồi nhưng mã HTTP nằm ngoài dải 2xx (ví dụ 404, 500), chương trình chặn đứng luồng và báo lỗi `HttpError`.
3. **Tầng Data (Phân tích cú pháp):** Kiểm tra tính toàn vẹn của JSON trả về bằng khối `try-catch (json::parse_error)`. Sau đó xác thực xem trường `message.content` có thực sự tồn tại và đúng định dạng chuỗi hay không (tránh lỗi trỏ null pointer).

3. Logic Ứng dụng Trực tiếp (CLI Main Loop)

File main.cpp không chỉ là môi trường kiểm thử mà được thiết kế như một phiên Chat liên tục, mô phỏng quá trình tương tác trực tiếp của người dùng với hệ thống:

```
// Logic vòng lặp chính
while (true) {
    // 1. Đọc dữ liệu thô (cho phép chứa khoảng trắng, enter)
    std::getline(std::cin, user_input);

    // 2. Kiểm tra điều kiện ngắt vòng lặp (exit / quit)
    // 3. Chặn rác (input rỗng)

    // 4. Giao tiếp an toàn qua std::expected
    auto result = llm.chat(text_prompt);

    // 5. Unpack kết quả & Đề xuất cách khắc phục khi có lỗi (ví dụ: nhắc bật Ollama)
}
```

4. Khả năng Mở rộng trong Tương lai

Với nền tảng kiến trúc vững chắc này, hệ thống sẵn sàng cho các nâng cấp tiếp theo của dự án:

- Tích hợp module đọc và băm (chunking) file PDF tiếng Việt để làm đầu vào (context) cho hàm `chat()`.
- Thêm logic Vector Database để truy xuất thông tin ngữ cảnh nhanh chóng.
- Mở rộng sang các API chuẩn OpenAI, vLLM bằng cách viết thêm một class kế thừa từ LLMClient mà không cần sửa đổi bất kỳ logic ứng dụng tầng trên nào.